

What I Talk About When I Talk About Platforms

Why an effective digital platform can help you scale delivery, what it should have in it, and how to get started building one.

These days everyone is building a 'platform' to speed up delivery of digital products at scale. But what makes an effective digital platform? Some organisations stumble when they attempt to build on top of their existing shared services without first addressing their organisational structure and operation model.

05 March 2018



Evan Bottcher

Evan is an experienced technical leader with more than two decades of experience building and integrating systems, providing clients with advice on technology strategy, enterprise architecture and software delivery practices.

CONTENTS

[What is a 'Platform' anyway?](#)

[First, an un-platform](#)

[The impact of “backlog coupling”](#)

[The half-arsed superficial private cloud](#)

[Autonomy speeds time to market, increases innovation](#)

[Technology diversification increases drag](#)

[Platform as an internal product](#)

[Wait a minute... isn't this a 'DevOps Team'?](#)

[Where do I start?](#)

🔗 [PLATFORMS](#)

(with apologies to Haruki Murakami.)

What is a 'Platform' anyway?

Words are hard, it seems. 'Platform' is just about the most ambiguous term we could use for an approach that is super-important for increasing delivery speed and efficiency at scale. Hence the title of this article, here is what I've been talking about most recently.

Definitions for software and hardware platforms abound, generally describing an operating environment upon which applications can execute and which provides reusable capabilities such as file systems and security.

Zooming out, at an organisational level a 'digital platform' has similar characteristics - an operating environment which teams can build upon to deliver product features to customers more quickly, supported by reusable capabilities.

A **digital platform** is a foundation of self-service APIs, tools, services, knowledge and support which are arranged as a compelling internal product. Autonomous delivery teams can make use of the platform to deliver product features at a higher pace, with reduced co-ordination.

At Thoughtworks we have developed a model with five key pillars of platform capability. These capabilities include delivery infrastructure, APIs and architectural remediation, self-service data, experimental infrastructure, and customer touchpoint technologies. We've learned through global experience that these are the most important shared capabilities to invest in to unlock the ability to become a digital company.

This article is focused on platform capabilities we'd classify as Delivery Infrastructure - including cloud hosting and devops tooling, although the same defining characteristics apply to other platform capabilities.

First, an un-platform

A couple of years ago I was engaged to do some consulting at a large Australian financial services organisation. Let's call them BigCo. My first goal upon arrival on-site

was to understand what was happening in the application infrastructure, hosting and operations area. To get a real understanding of where the challenges were at, we decided to follow a real change through the system of work and see how things were done.

Despite a big investment in cloud and automation, BigCo retained a traditional arrangement of teams in the infrastructure and operations area. Teams were divided along technical competencies. We followed a few typical changes, and they each involved a number of these teams. If there is a change to the application server configuration, this is owned by the 'middleware' team. The middleware team don't have access to the underlying operating system configuration however, that is the responsibility of the 'midrange' team. Database changes must be done by the DBA team. Networking changes must go through a networks team. Load balancer changes must be done by a managed service provider, while firewall changes are done by a different provider. There was also a separate automation team, who owned some automation capability - mostly limited to orchestration. There was of course separate enterprise monitoring, security, change and release management teams.

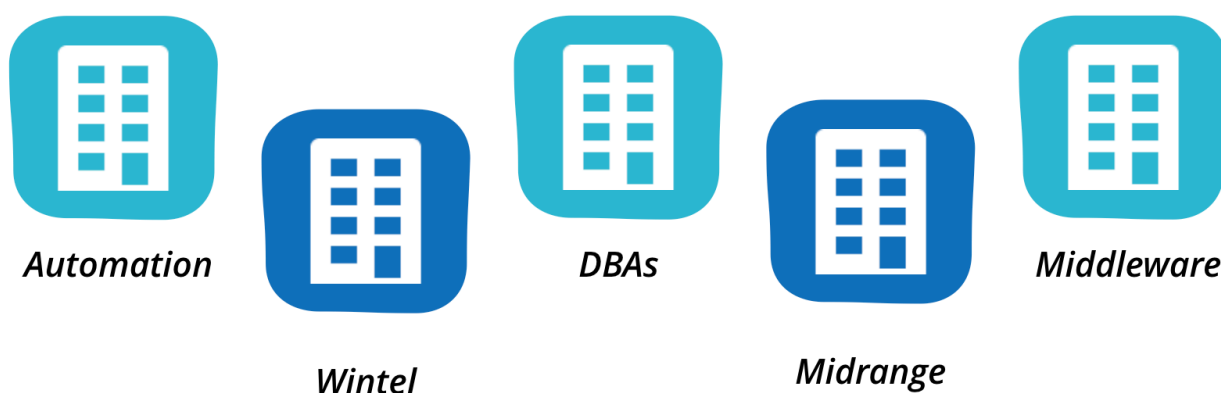


Figure 1: Separate highly specialised infra and operations teams

Each of these BigCo teams were under their own management structure, with independent ways of working. Each team was managing towards a high degree of efficiency in their own technical domain, centralising specialisation, outsourcing non-differentiating capabilities, applying governance, and reducing costs. However no-one at BigCo was measured on effectiveness of end-to-end delivery of features to customers.

Small changes involving infrastructure took anything from several weeks to multiple months, with a huge impact to responsiveness to the customer. This has a big impact, but that's not all. We notice that when change is hard and slow, then any failures in the change process cause further delays. This drives a behaviour in engineers and managers to reduce to a minimum the number of changes made as much as possible, only making the absolutely essential changes to applications and infrastructure.

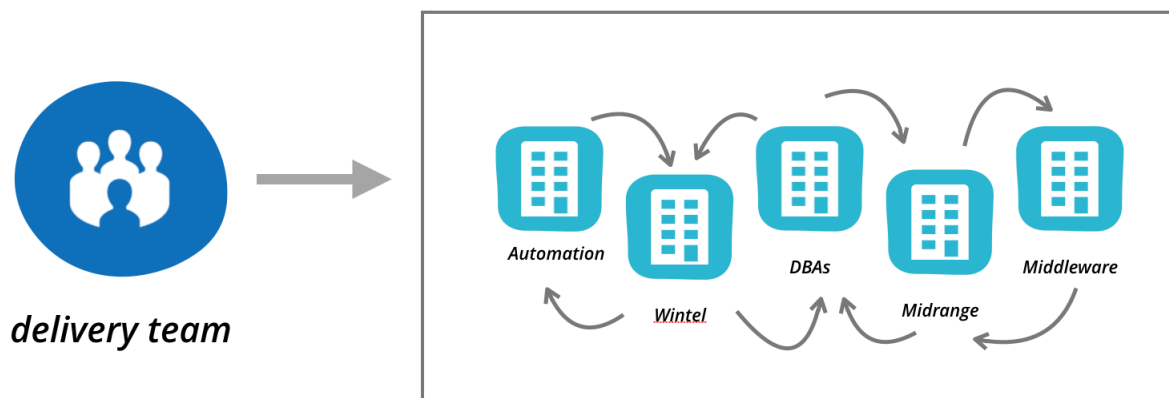


Figure 2: Changes required by application delivery teams would take weeks or months

At BigCo this had clearly led to a gradual decline in the internal quality of applications and infrastructure - with lots of little inconsistencies in environments and configuration settings everywhere. The teams had stopped making the small improvements and refactorings that would maintain or improve quality and consistency. This is self-reinforcing in practice: as quality impacts predictability and therefore increases risk of change, so teams become more cautious and it becomes harder to make improvements.

So in summary: dealing with infrastructure and hosting at BigCo was slow and difficult.

The impact of “backlog coupling”

The low hanging fruit for agile software delivery has always been in the digital channels, with small autonomous teams working closely with business leaders identifying customer need and building features that meet those needs. However the faster and more responsive a digital product team becomes, the more amplified the external constraints applied to it become.

Digital teams are generally constrained in three main areas: slow-moving core transactional systems of record, access to high-quality data and analytics, and shared infrastructure and operations.

I use the term ‘backlog coupling’, where backlog is a planning tool often used by agile delivery teams.

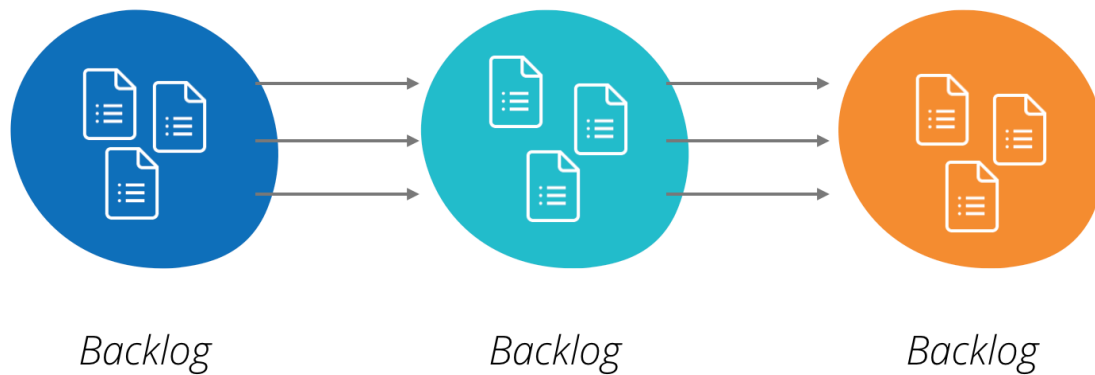


Figure 3: Backlog coupling happens when changes have dependencies across the backlogs (work queues) of multiple teams

It's a simple concept – if a large number of items in the backlog of a digital product team require a corresponding backlog item to be raised in another team, then productivity and responsiveness suffer enormously. Backlogs get chained across the organisation, each being prioritised according to a different system. Tasks acquire big red ‘blocked’ stickers on kanbans, stakeholders get angry, the shared service providers react as best they can – usually responding to the loudest voice.

How bad is backlog coupling? At an Australian telecommunications company, my colleagues did a study of hundreds of pieces of work or tasks passing through a delivery centre. Some tasks could be completed by a single team without dependency, specifically without scheduling work by members of another team. The tasks that had to wait for another team were 10-12x slower in elapsed time. So dependencies have a real significant impact.

This hurts us in many ways: it hurts in pure throughput and responsiveness to customer need, and drives us towards more long-term planning to more efficiently manage dependencies. It also damages a team's own accountability for outcomes, and for many teams I've observed this is a motivation-killer. Teams can find it easy to shift blame and stop seeking their own continuous improvement.

It's not much fun to be in one of the heavily overloaded teams servicing many noisy internal customers either.

The recent tradition of 'scaling agile' tries to solve this in one way - by introducing planning ceremonies that attempt to align priorities across multiple teams. This explicitly trades off an increase in alignment for overall reduced autonomy, responsiveness and ability to respond to change. This cannot be the only way.

So clearly a characteristic of a good platform must be that it reduces the amount of backlog coupling. The platform must provide services that do not require tickets to be raised and work to be assigned. *Self-service* is a key defining characteristic for a good platform.

The platform should provide a team with self-service access. Specifically it should allow for self-service *provisioning*, self-service *configuration*, and self-service *management and operation* of the platform capabilities and assets.

The ~~half-arsed~~ superficial private cloud

BigCo recognised the need for self-service, but struggled to envision how to achieve this with a large infrastructure and operations organisation entrenched in legacy infrastructure and thinking. There was already an investment in centralised automation tooling, so the first effort was to create a self-service capability for application delivery teams to self-provision infrastructure.

A self-service tool was built to allow delivery teams to requisition compute instances according to a very fixed template. The virtual machine instances provisioned were fixed in configuration and securely locked down to ensure control of the fleet could remain with the central midrange team. In order to do something useful with the instance - for example install packages, connect it to a network, attach storage, provision a load balancer, configure monitoring tools, or anything else - the delivery team would need to raise tickets.

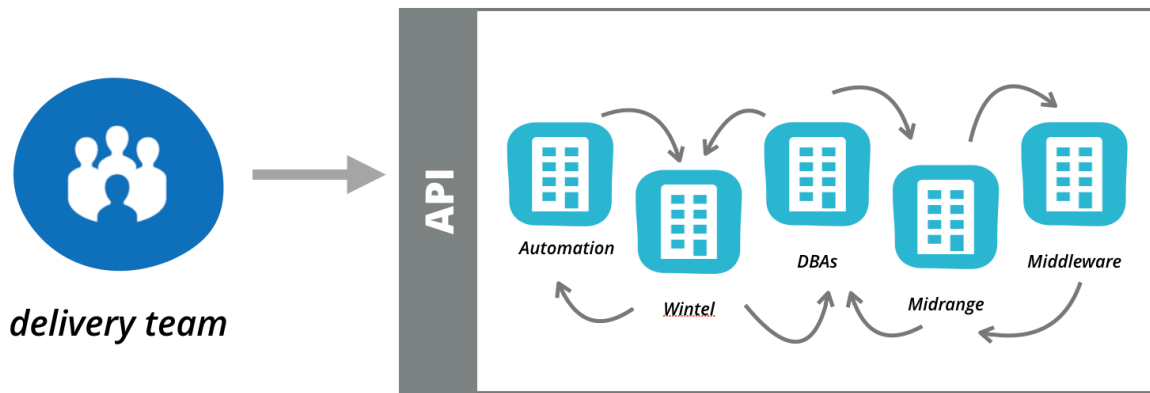


Figure 4: BigCo built a rudimentary self-service API without altering the fundamentals of how applications and infrastructure was run. The result did not significantly change the pace of delivery.

You could argue this was just the first iteration, and that was true - however the intent was clear. The BigCo infrastructure and operations group was not ready at that time to break down its own organisational silos, and shift significant responsibility (and therefore access) to delivery teams. And even if the intent was good, the sheer amount of effort required to incrementally build that API to the required richness was not viable.

We call this type of approach a ‘superficial private cloud’ - re-labelling existing virtualisation platforms for use by delivery teams in a very constrained way, with no real intent to reduce the amount of centralised control.

Meanwhile at BigCo in a parallel effort a delivery team had unlocked the ability to directly use Amazon Web Services (AWS) for production systems. Once the precedent was in place, delivery teams joined a stampede to use AWS.

AWS is a compelling platform for direct consumption by delivery teams: it is completely self-service, and has clear boundaries of responsibility. “You build it, you run it” becomes the mantra, where Amazon builds and runs the platform up to the API and makes sure it is highly available, and your application delivery team builds, configures, and runs applications that sit on top.

Is that the end of the story?

Autonomy speeds time to market, increases innovation

Most organisations I meet have a default setting of “build for re-use”: centralisation of capabilities driven by a combination of risk-aversion and cost reduction.

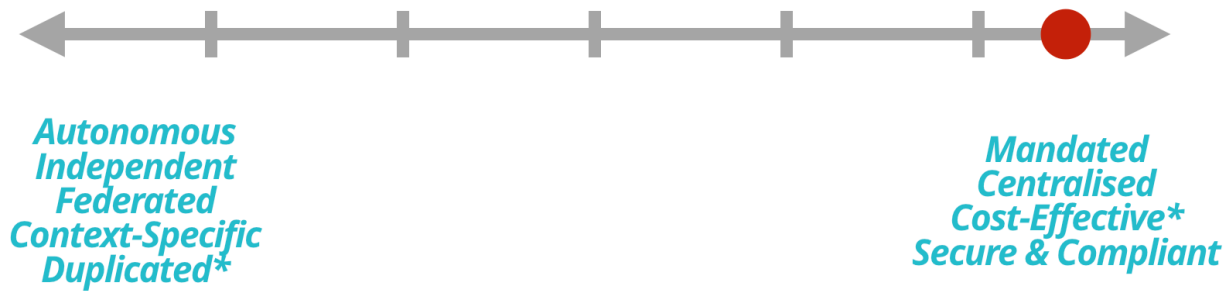


Figure 5: Most organisations default to centrally mandating technology choices to gain cost efficiency

In the last few years I was fortunate enough to be part of a tech leadership team at a major Australian (and global) technology company with a huge online presence, let's call them WebBiz. With multiple hundreds of engineers, they're big enough to face many of the same kind of challenges as BigCo, with a not-insignificant legacy in infrastructure and applications and data - but WebBiz is just small enough to see rapid change and improvement.

While I was at WebBiz, we made a start on a multi-year migration from deploying most applications on a virtualisation platform in a leased data centre to a new default deployment target of AWS. We also shifted responsibility for both build and run of applications and (most) infrastructure out to product teams, in the most complete transition from traditional central ops to devops I've witnessed. I believe it's actually not that hard to start a small organisation with a "you build it, you run it" mindset, but making the transition takes courage and continuity of vision. WebBiz have done very well in that regard.

As part of the migration, product teams at WebBiz were given complete autonomy over how they configured and ran every part of their stack. The approach was named 'Team Managed Infrastructure' - although there were some defaults established early on, each team would make it's own decisions on every part of the stack with almost no central mandates.

WebBiz had successfully reversed the typical default bias for organisations: favouring technology diversification and invention. This drove up the level of staff engagement, with engineers getting experience deeper in the stack of technology, drove invention, quickly established a level of responsibility for what was deployed, and eliminated the bulk of team dependencies. It also attracted engineers to work at WebBiz who are

interested in both taking on responsibility for what they run, who respond well to autonomy, and who are interested in solving difficult business and technical problems.

Technology diversification increases drag

However, for all the benefits there is a definite cost to shifting to full autonomy. By adopting AWS as a platform, WebBiz has removed the backlog coupling to the centralised infrastructure team. However every team at WebBiz is now forced to make a series of decisions around every aspect of building and operating their infrastructure.

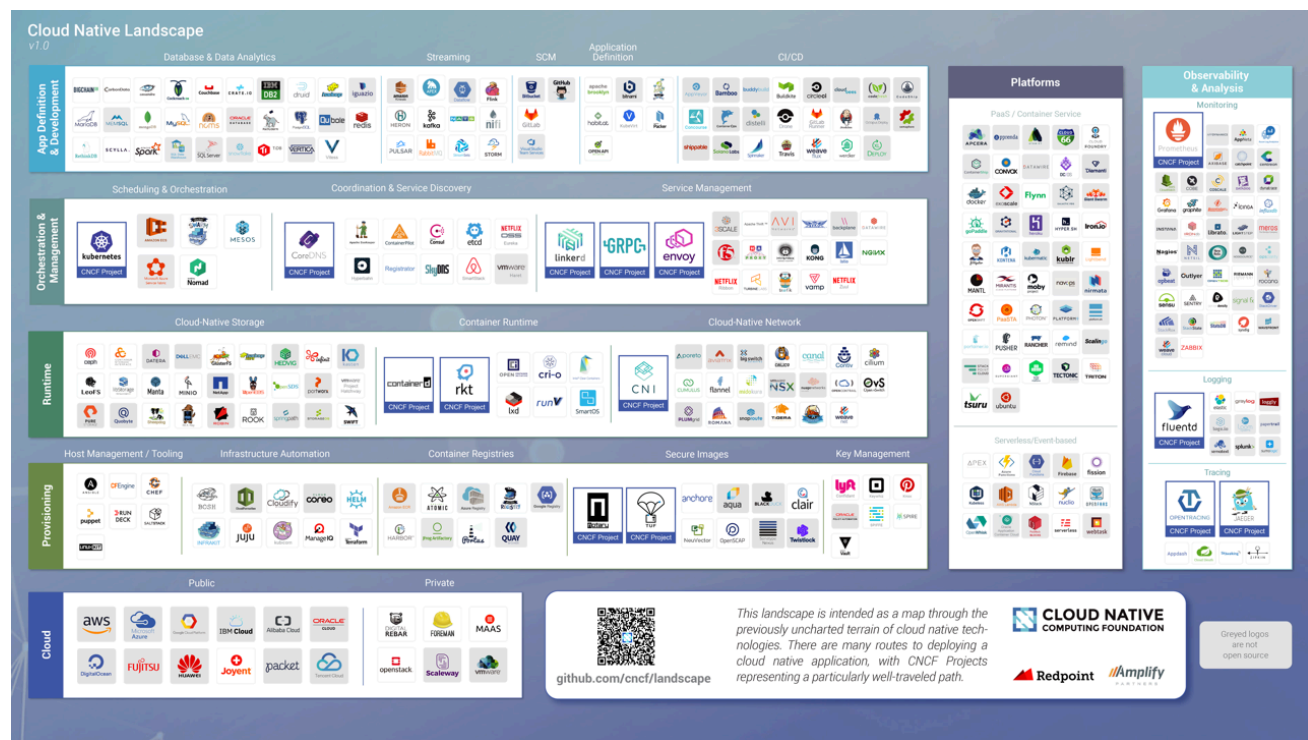


Figure 6: Cloud Native Landscape (source: www.cncf.io)

Above is a recent version of the Cloud Native Landscape: an attempt to capture some common open source and product offerings grouped by areas of concern in building a cloud native architecture. It's a crowded map, and it's only the most well established offerings. For each of these areas and more, a team must evaluate options and choose an offering that is a good fit to their needs, then learn how to integrate and operate that offering.

In addition to the maintenance drag of duplicated infrastructure, there is an ongoing overhead for each team to continually research and evaluate their infrastructure choices. There's also a friction created which works against the transfer of skills and even staff between teams which are running distinctly different stacks.

WebBiz is now beginning to establish a more-clearly defined delivery infrastructure platform - a compelling set of defaults that product teams can consume to reduce the amount of drag and increase their productivity.

But do they risk losing the benefits that autonomy has brought?

Platform as an internal product

It is a real struggle to find the right balance between autonomous diversification and mandated consolidation, and that is even more difficult to predict up front. A key ingredient for success in finding this balance is that platforms must be compelling to use, they cannot stand on a mandate alone. Your existing shared infrastructure function has a monopoly and delivery teams have no viable alternative. A little competition is a necessary ingredient to drive the product thinking ensuring that every platform feature adds value instead of creating constraints and coupling.

A key ingredient for success in finding this balance is that platforms must be compelling to use

What makes a platform compelling? Here are a few ideas:

- The platform is self-service for the overwhelming majority of use cases.
- The platform is composable, containing discrete services that can be used independently.
- The platform does not force an inflexible way of working upon the delivery team.
- The platform is quick and cheap to start using, with an easy on-ramp (e.g. Quick start guides, documentation, code samples)
- The platform has a rich internal user community for sharing
- The platform is secure and compliant by default
- The platform is up to date

Ultimately the delivery infrastructure platform is compelling when it is easier to consume the platform capability than building and maintaining your own thing. Netflix describes their centralised tooling as ‘the paved road’ - teams do not have to make use of the tooling but are responsible for all the costs of maintaining their own alternatives.

A platform should also be more than just software and APIs - it is documentation, and consulting, and support and evangelism, and templates and guidelines.

Wait a minute... isn't this a 'DevOps Team'?

Done badly? Yes it might be.

We totally lost the whole "DevOps" isn't a role/team/tools" battle didn't we? We keep losing these battles. Maybe a new strategy next time?

-- Phil Calçado

(I'm not ready to admit defeat just yet on DevOps: so just in case you aren't sure, if you have a team called 'DevOps' then that word doesn't mean what you think it means.)

You may choose to build a team to build and operate a delivery infrastructure platform - I think in most cases that's going to be the best way to get started. If so, you should be very clear on the scope of responsibilities for the platform team vs its customers - which I'll call application teams for clarity.

Application teams build, deploy, monitor, and are on call for the application components and application infrastructure that they provision and deploy on the platform. Platform teams build, deploy, monitor, and are on call for the platform components and underlying platform infrastructure.

The platform team ideally doesn't even know what applications are running on the platform, they are only responsible for the availability of the platform services themselves.

In this way both application teams and platform teams have responsibility for build and run of their own products. 'You build it, you run it' still applies.

Where do I start?

There are some prerequisites for success in establishing a delivery platform. Firstly you will probably already be on your journey to move away from 'project' as the primary mechanism for funding and staffing delivery of technology. Platform is a product, and needs a long-lived and stable product team tasked with both build and run.

Secondly you must be willing to shift some or all of the run responsibility for applications into the application teams and away from centralised operations and support. The platform provides tools and services to allow application teams to take responsibility for what they build, this won't happen while support is centralised.

Thirdly you must be willing to trade off strict consistency of implementation against the freedom and responsibilities that you're handing to the autonomous application teams.

Now some gotchas.

- Your platform is not only infrastructure and tools and APIs that you can install. To be effective you must answer how your delivery teams will quickly adopt the new capabilities, what choices will they make independently vs using sensible defaults, and how you will maintain the capabilities ongoing. This will require some internal consulting skills and training and evangelism.
- You do not know what platform capabilities you need, so start small based on genuine proven needs. Harvest already proven solutions from application teams, and try joint-ventures to create and test capabilities with the teams that will use them.
- Be very careful not to just apply the platform label to the limited virtualised hosting and locked-down centrally-managed tooling that you already have.



► Significant Revisions